OPERATING SYSTEMS CSCI-SHU215

Lecture 05 – Traps

Purpose

Prevent polling

- Event notification
  - *eg. device ready, I/O return, error, ...*
- Associated notion: DMA (Direct Memory Access)
  - I/O management via hardware controller
    - ie. CPU does not get involved

Protect the system

- 2 execution modes: user / kernel
- System call => trap handler in kernel mode

Allow **multi-programming**

- Concurrent access to CPU
  - => Context switching
  - Periodic interrupt triggered by the clock



Main definitions

A trap is a temporary suspension of the normal execution of a process to handle a particular event.

An interrupt service routine (ISR), or interrupt handler, is the computation associated with handling an event.

More terminology

Trap

Any kind of a control transfer from the user to the OS

Interrupt

Device-initiated

e.g. Clock tick, network packet, I/O completion

Exception

Program-initiated in response to an exceptional event

e.g. Divide by zero, segmentation fault

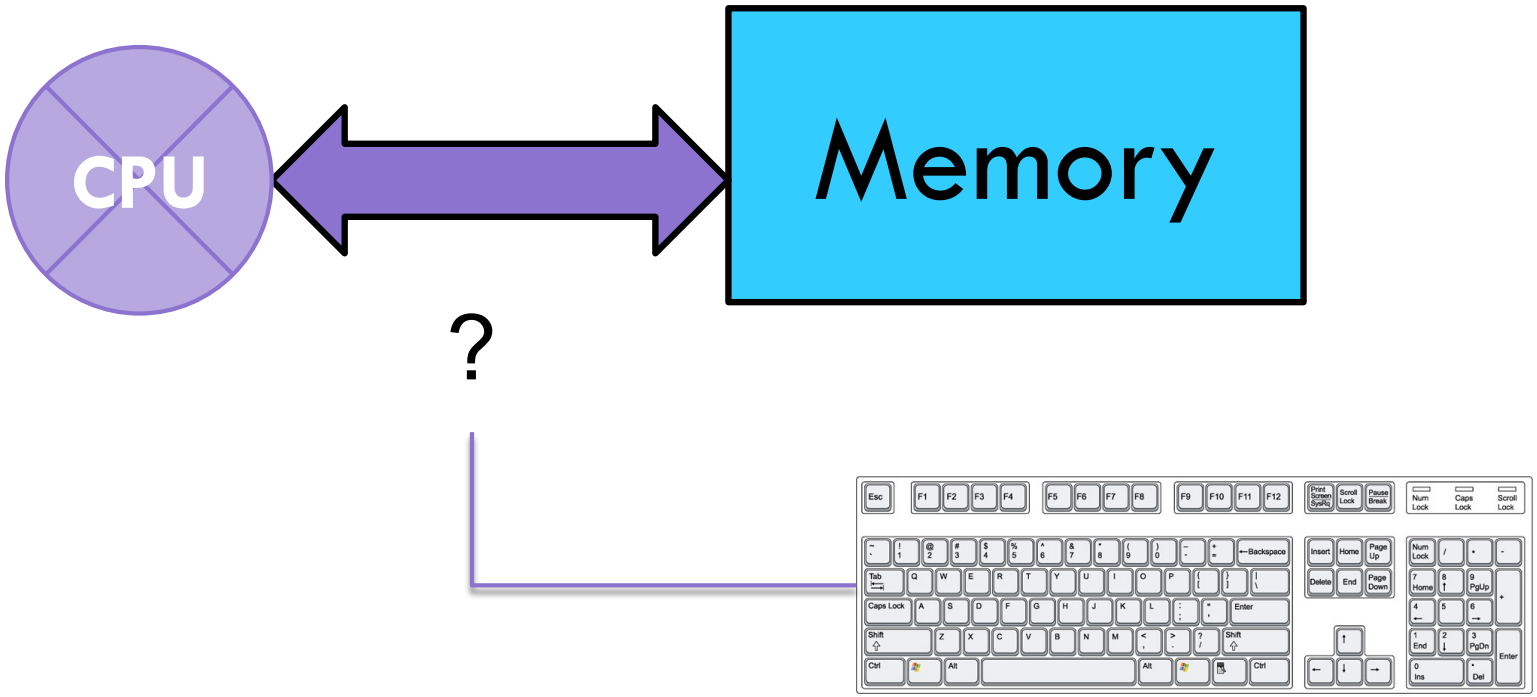
SYSCALL

Program-initiated as means to obtain a service from the OS

e.g. System call

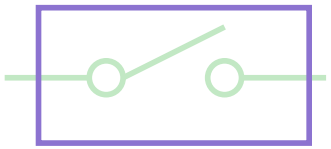
Keyboard I/O Example

Keyboard I/O example

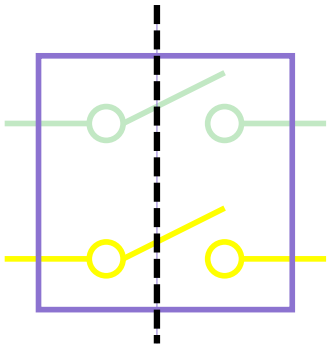


© E. G. Sirer - Cornell U.

Keyboard I/O example



SPST



DPST

Let's build a keyboard

- Reading input

  Mechanical switches

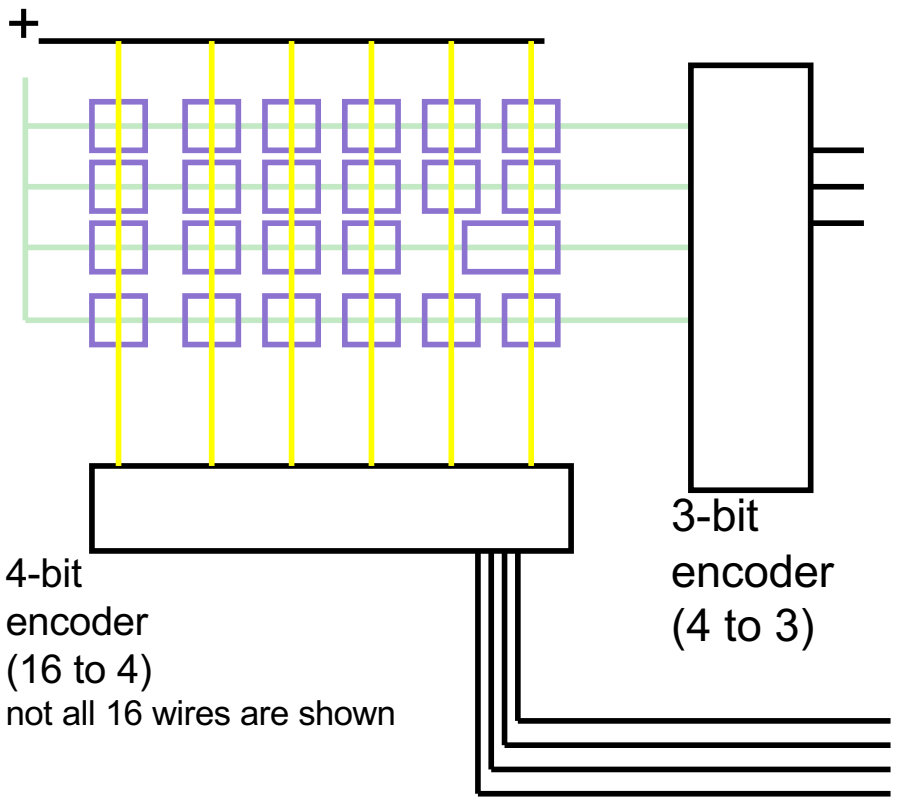
  - Single pole, single throw
  - Double pole, single throw

- Digital processing

  Encoders, decoders, muxes, latches, tristate buffers, logic gates, …

© E. G. Sirer - Cornell U.

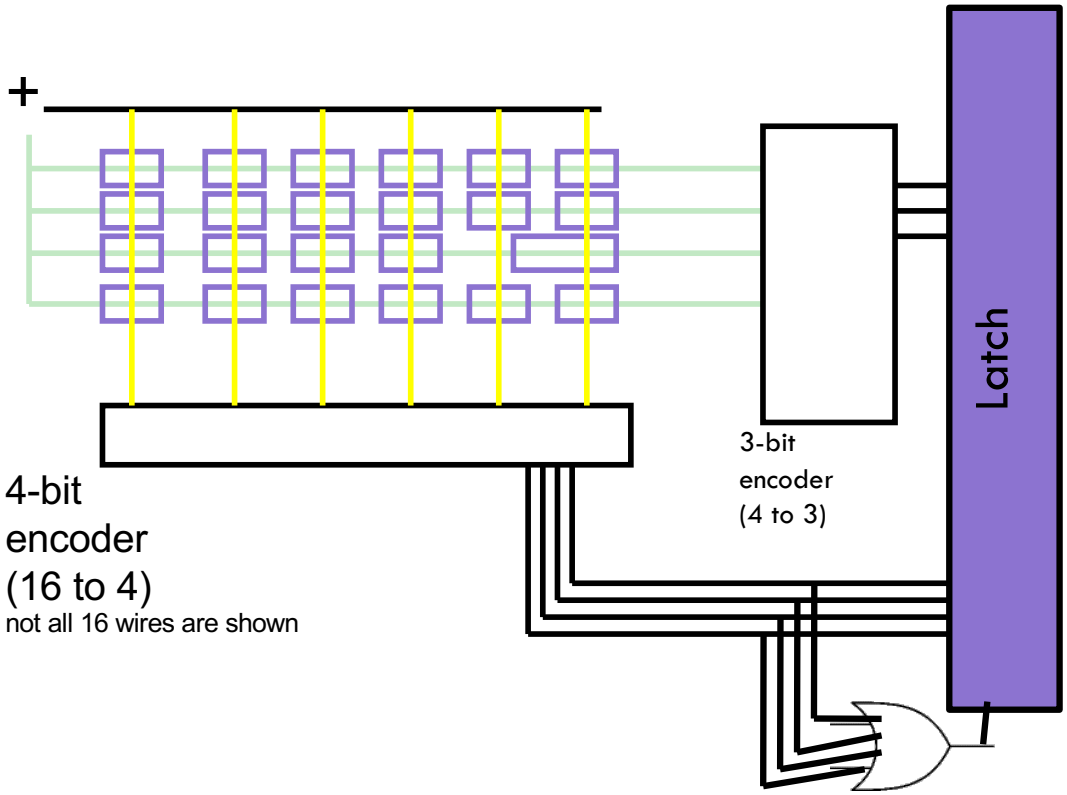
Keyboard I/O example



When a key is pressed, a 7-bit key identifier gets computed

© E. G. Sirer - Cornell U.

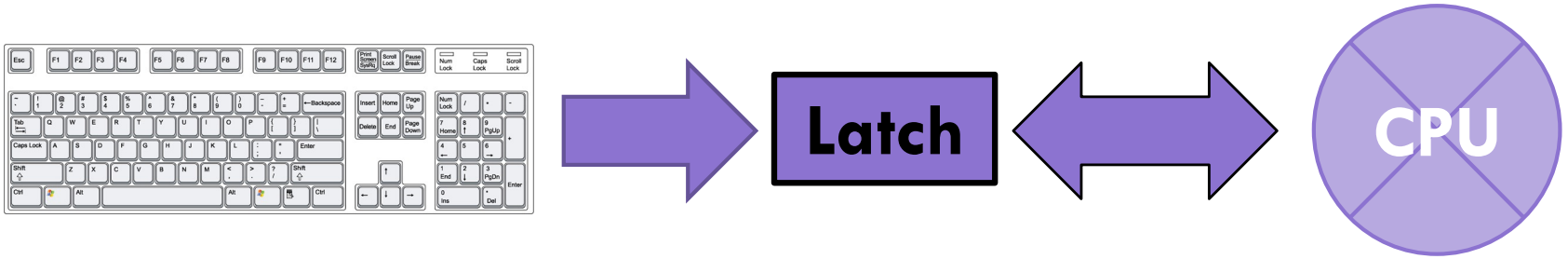
Keyboard I/O example



A latch can store the keystroke indefinitely

© E. G. Sirer - Cornell U.

Keyboard I/O example



A **latch** can store the keystroke indefinitely

The keyboard can then appear to the CPU as if it is a special memory address

In this design, the CPU *constantly* needs to read the keyboard latch memory location to see if a key is pressed - **Polling**

© E. G. Sirer - Cornell U.

Polling vs. Interrupts

Polling is inefficient

Event notification alternative: **interrupt driven I/O**

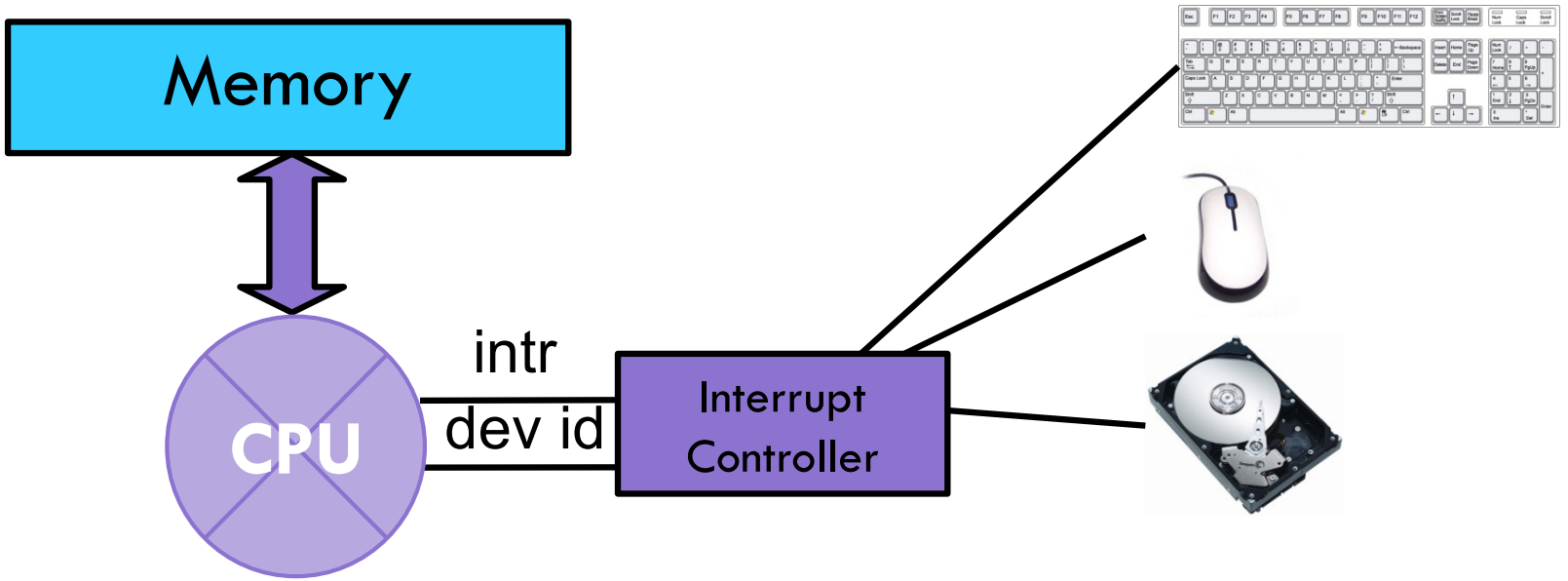
- add extra circuitry
- the keyboard alerts the CPU when there is a keypress

Enables the CPU and devices to perform tasks concurrently

Increases throughput

© E. G. Sirer - Cornell U.

Interrupt Driven I/O



An interrupt controller mediates between competing devices

1. Raises an interrupt flag to get the CPU's attention
2. Identifies the interrupting device

Can disable (aka mask) interrupts if the CPU so desires

© E. G. Sirer - Cornell U.

Interrupt Management

Interrupt controllers manage interrupts

- Maskable interrupts: can be turned off by the CPU for critical processing
- Non-maskable interrupts: operation-critical errors (e.g. unrecoverable memory error, power out warning, etc)

Interrupts contain a descriptor of the interrupting device

A priority selector circuit examines all interrupting devices

Reports highest level to the CPU

Interrupt controller implements interrupt priorities

Can optionally remap priority levels

© E. G. Sirer - Cornell U.

Direct Memory Access (DMA)

Transfers data directly between device and memory

- No CPU intervention required for moving bits
- Device raises interrupts solely when the block transfer is complete

Critical for high-performance devices

© E. G. Sirer - Cornell U.

Interrupt-driven I/O summary

Normal interrupt-driven operation with memory-mapped I/O proceeds as follows

- CPU initiates a device operation (e.g. read from disk) by writing an operation descriptor to a device register
- CPU continues its regular computation
- The device asynchronously performs the operation

  Device controller / DMA controller interaction
- When the operation is complete, DMA controller interrupts the CPU

© E. G. Sirer - Cornell U.

Context Switching

Context switch

Cornerstone of multi-programming

Eviction of the running process to the benefit of another process

1. Suspend the running process (P)
2. Store the context of P
3. Run ISRs (eg. update stats & clock)

   Interrupt Vector Table maps each interrupt to a handler
4. Elect a process Q
5. Load the context of Q

Context switch

Process Control Block (PCB)

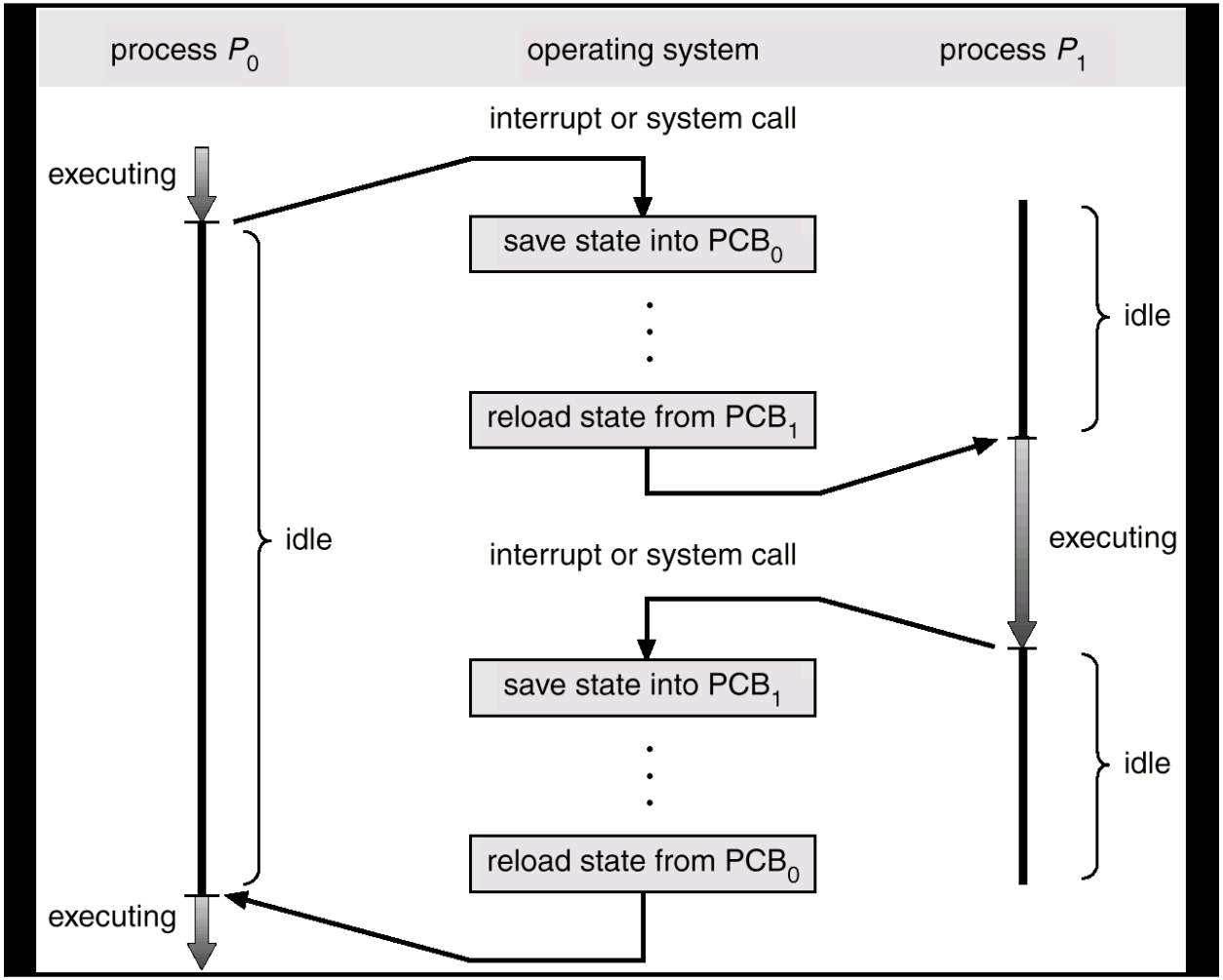
Data structure associated with each process

Describes the execution context of the process

Typically contains:

- PID (process ID)
- Register values, including the Program Status Word
  - status (active or standby)
  - mode (user or kernel)
  - interrupt mask
  - PC (in some systems)
- Info about scheduling (eg. priority)
- Information about I/Os (eg. open files)
- Statistics (CPU time, date of insertion, ...)

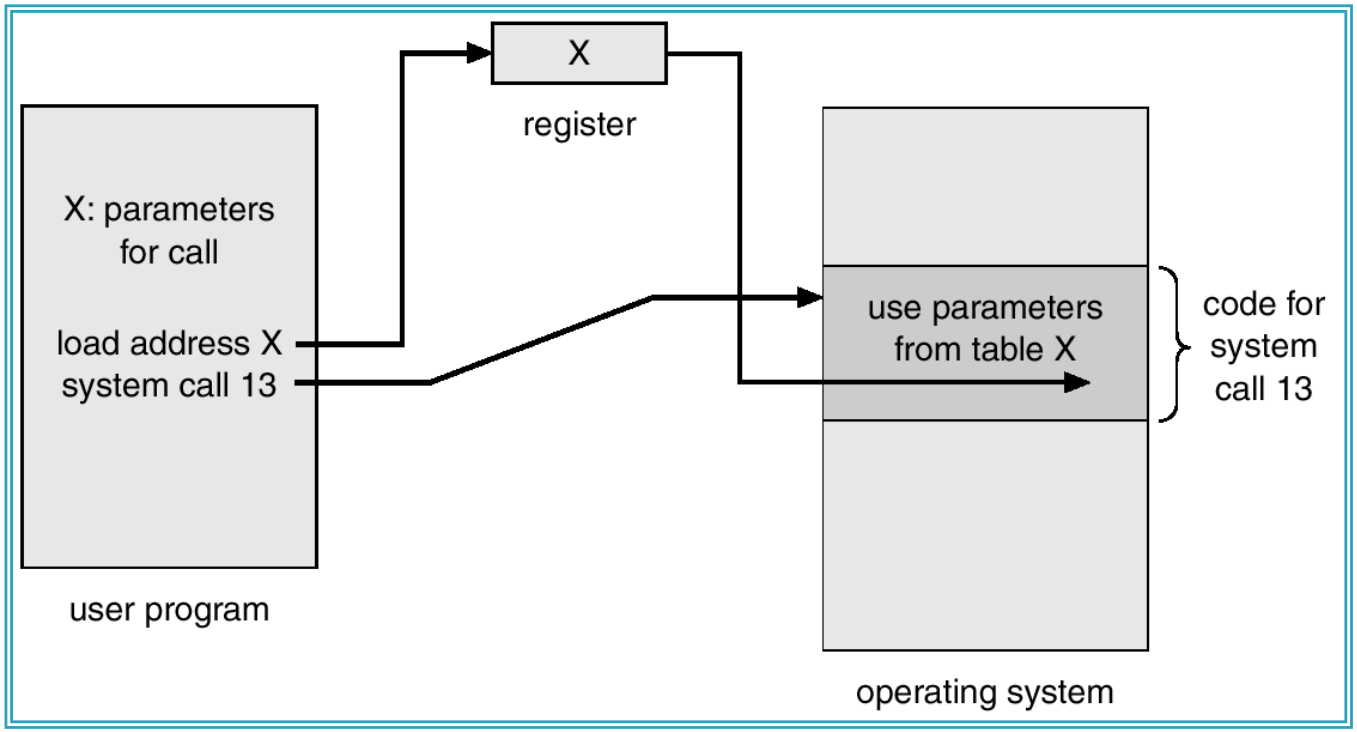
Context switch



System Calls

Controlled transfer of execution from unprivileged code to the OS

SYSCALL instruction transfers control to a system call handler at a fixed address



SYSCALL instruction

SYSCALL instruction does an atomic jump to a controlled location

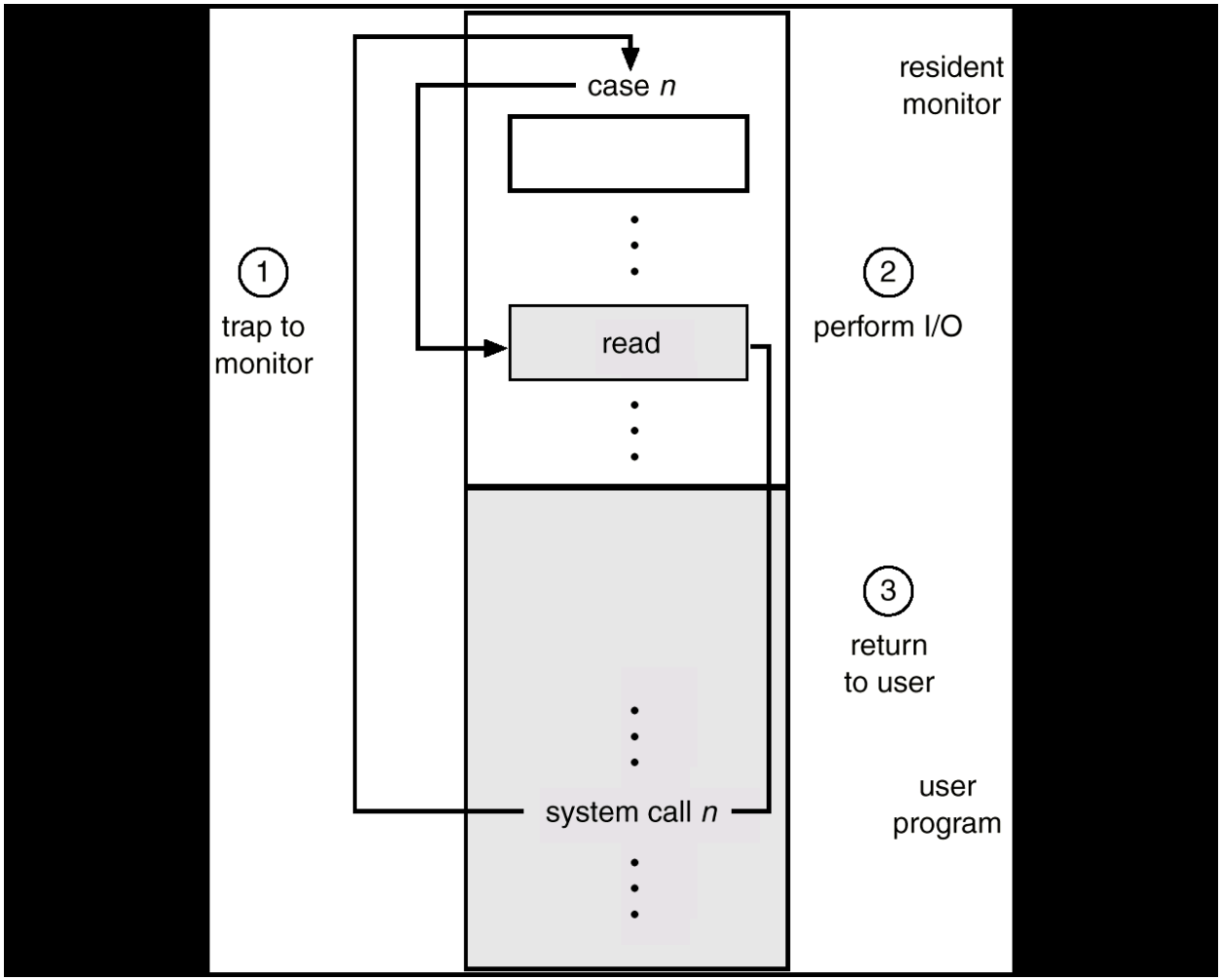
- Switches the stack pointer (SP) to the kernel stack
- Saves the old (user) SP value
- Saves the old (user) program counter (PC) value (= return address)
- Saves the old privilege mode
- Sets the new privilege mode to 1
- Sets the new PC to the kernel syscall handler

Kernel system call handler carries out the desired system call

- Saves caller-save registers
- Examines the syscall number
- Checks arguments for sanity
- Performs operation
- Stores result in v0
- Restores caller-save registers
- Performs a "return from syscall" instruction, which restores the privilege mode, SP and PC

© E. G. Sirer - Cornell U.

System call


resident
monitor
case n
read
system call n
1
trap to
monitor
2
perform I/O
3
return
to user
user
program

Libraries and Wrappers

Compilers do not emit SYSCALL instructions

They do not know the interface exposed by the OS

Instead, applications are compiled with standard libraries, which provide "syscall wrappers"

printf() -> write(); malloc() -> sbrk(); recv(); open(); close(); …

Wrappers:

- are written in assembler,
- internally issue a SYSCALL instruction,
- pass the arguments to the kernel,
- pass the result back to the calling application

© E. G. Sirer - Cornell U.

Exceptional Situations

System calls are control transfers to the OS, performed under the control of the user application

Sometimes, need to transfer control to the OS at a time when the user program least expects it

- Division by zero,
- Alert from the power supply that electricity is about to go out,
- Alert from the network device that a packet just arrived,
- Clock notifying the processor that the clock just ticked,

Some of these causes for interruption of execution have nothing to do with the user application

Need a (slightly) different mechanism, that allows resuming the user application

© E. G. Sirer - Cornell U.

Interrupts & Exceptions

On an interrupt or exception

- Switches the sp to the kernel stack
- Saves the old (user) SP value
- Saves the old (user) PC value
- Saves the old privilege mode
- Saves cause of the interrupt/exception
- Sets the new privilege mode to 1
- Sets the new PC to the kernel interrupt/exception handler

Kernel interrupt/exception handler handles the event

- Saves all registers
- Examines the cause
- Performs operation required
- Restores all registers
- Performs a "return from interrupt" instruction, which restores the privilege mode, SP and PC

© E. G. Sirer - Cornell U.

SYSCALL vs. Interrupt

The differences lie in how they are initiated, and how much state needs to be saved and restored

SYSCALL requires much less state saving

- Caller-save registers are already saved by the application

Interrupts typically require saving and restoring the full state of the processor

- Because the application got struck by a lightning bolt without anticipating the control transfer

© E. G. Sirer - Cornell U.